

AuditLens

Statutory audit analytical review for Indian companies. A trial balance goes in; the face of the Schedule III financial statements, the eleven ratios MCA made mandatory in 2021, journal entry testing under SA 240, a monetary unit sample under SA 530 and the twenty-one clause CARO 2020 checklist come out — as a working web application and an eleven-sheet Excel workpaper.

Submitted as the Module C capstone project for **AICA Level 2** (AI for Chartered Accountants, ICAI), Batch 104.

The governing rule

The engine computes. The model writes prose. Nothing else.

Every figure that reaches a workpaper is produced by a pure Python function with a unit test. The language model is asked for two things only: an explanation in words, and correspondence. It never computes a figure, never classifies a ledger that reaches the face of the statements, and never concludes on a CARO clause.

A Chartered Accountant signs the report. Every number in it has to be traceable to a calculation that can be re-performed.

What it does

	Module	Authority
1	Reads a trial balance and general ledger in whatever shape the client's system exported them — Indian number formats, aliased headers, duplicate codes	—
2	Maps every ledger to a Schedule III Division I presentation head, on the account code first and the ledger name second	Schedule III, Companies Act, 2013
3	Builds the Balance Sheet and Statement of Profit and Loss, and reports honestly when they do not tie	Schedule III, Division I
4	Computes the eleven mandated ratios and flags every movement beyond 25 per cent	G.S.R. 207(E) dated 24 March 2021
5	Runs six journal entry routines and the Benford first-digit test	SA 240
6	Determines materiality from the auditor's benchmark, and selects a monetary unit sample with a recorded seed	SA 320, SA 530
7	Tests CARO applicability and pre-populates the twenty-one clauses from the books	CARO 2020
8	Drafts the analytical memorandum, the ratio variance notes and the enquiry letter	—

Full authority for each, and what the tool deliberately does not do, is in `docs/statutory_basis.md`.

Reading these documents on a laptop. Markdown (`.md`) is the right format for GitHub but has no default application on most Windows machines — double clicking one often opens Adobe Acrobat, which reports the file as damaged. PDF copies of every document are in the repository: `AuditLens.pdf` ,

PROJECT_SUMMARY.pdf , and docs/*.pdf . Rebuild them with `python docs/build_pdfs.py` .

Run it

Start with `START_HERE.txt` in this folder — it opens in Notepad on any machine and carries both routes below.

The quickest way

Windows — double-click `run-windows.bat` .

If double-clicking opens the file in Notepad rather than running it, the `.bat` association on that machine points at a text editor. Don't fight it — use the typed route below, which works regardless and films better anyway. If Windows blocks the file because it came from the internet, right-click → **Properties** → tick **Unblock**. And extract the folder from the ZIP first; a batch file will not run from inside a zip preview window. **macOS or Linux** — run `./run.sh` in a terminal.

Either one sets up a private environment on first use (a minute or two), starts the server, and opens your browser **once the server is actually listening** — not before. On later runs it starts in a few seconds. Close the window to stop it.

If port 8000 is already taken by something else, it moves to 8001 and says so in the window; the browser it opens points at the port it actually used.

Once the page is open, click **Use the sample client** — it runs the whole review on the bundled synthetic company without you supplying anything.

You need Python 3.10 or later installed. On Windows, tick "**Add python.exe to PATH**" during the Python installer, or the batch file cannot find it.

The manual way

```
python -m venv .venv
.venv\Scripts\activate           # Windows
source .venv/bin/activate        # macOS or Linux

pip install -e "[dev]"

python -m auditlens.launch        # the web application, opens the browser for you
auditlens --samples               # or the command line, straight to a workpaper
```

The command line

Useful when you want the workpaper without the interface:

```
# The bundled synthetic client
auditlens --samples --out workpaper.xlsx

# A real engagement
auditlens \
  --tb trial_balance_2024-25.xlsx \
  --prior trial_balance_2023-24.xlsx \
  --gl general_ledger.csv \
  --client "Acme Manufacturing Private Limited" \
  --fy 2024-25 --year-end 2025-03-31 \
  --credit-sales-ratio 0.9 --principal-repayments 5000000 \
  --drafts --out acme_workpaper.xlsx
```

`auditlens --help` lists every option.

What your files need to contain

Trial balance — one row per ledger, with columns for the account code, the ledger name, and the debit and credit balances. Common header spellings are recognised automatically (`Ledger Code` , `Particulars` , `Dr` , `Cr` and others), as are Indian number formats, rupee signs and bracketed negatives.

Account Code	Ledger Name	Debit	Credit
2251	Trade receivables	4,92,80,000	
1332	Trade payables		3,46,20,000

General ledger — one row per journal line, with `entry_id` , `posting_date` , `account_code` , `account_name` , `debit` , `credit` , `narration` and `posted_by` . An `entry_date` column, where your client's system records one, enables the back-dated entry routine.

Look at the files in `samples/` for the exact shape.

Drafting

Drafting works offline out of the box, using deterministic templates — no key, no network. To use the model instead:

```
set GEMINI_API_KEY=...           # Windows
export GEMINI_API_KEY=...        # macOS or Linux
export AUDITLENS_MODEL=gemini-2.0-flash # optional
```

The analytical result is identical either way, because the model never touches a figure.

Tests

```
python -m pytest
```

124 tests. One file per area of statutory logic, and every one of the eleven ratios checked against a hand-computed figure — so a change that would alter a disclosed ratio fails here rather than in a client's financial statements.

Two examples of what the suite is for:

- `test_balance_sheet_ties_once_profit_and_unmapped_are_dealt_with` caught a real bug during development: profit was dropped for a client with no reserves ledger.
- `test_flag_rate_stays_proportionate` asserts that no SA 240 routine flags more than 10 per cent of the population. A routine that flags everything is useless to an engagement team, and that is a defect the arithmetic alone would not reveal.
- `test_the_browser_opens_only_after_the_port_answers` is a regression test. The first Windows launcher opened the browser before the server had bound its port, so the first thing anyone saw was `ERR_CONNECTION_REFUSED` while the server came up five seconds behind it.

What the sample client demonstrates

`samples/generate.py` builds **Bharat Precision Components Private Limited**, a fictitious manufacturer. No client data is used anywhere in this project.

Defects are seeded so every routine has something real to find:

Seeded	Found by
A ledger outside the firm's numbering convention	The keyword fallback — mapped, and queued for review
A suspense account	Returned <code>UNMAPPED</code> ; the balance sheet is then out by exactly that amount
A sharp profitability improvement	Four ratios beyond 25 per cent
Round-sum, weekend, holiday, back-dated, period-end and rare-combination entries	The six SA 240 routines
A user who posts almost nothing	The infrequent-user routine

Transaction amounts are drawn log-uniformly across two decades, because real transaction populations are scale-invariant and conform to Benford's law. A uniform draw would not, and the Benford routine would report a false departure on a clean population.

Repository

```
auditlens/
├─ START_HERE.txt      how to start it, in plain text
├─ run-windows.bat     one-click start on Windows
├─ run.sh              one-command start on macOS and Linux
├─ AuditLens.pdf       this README, as a PDF
├─ PROJECT_SUMMARY.pdf the two-page summary, as a PDF
├─ presentation/       the capstone deck, with speaker notes, and its screenshots
├─ prompts/           four versioned system instructions, with changelogs
├─ src/auditlens/
│   ├─ ingest.py       reading and validation
│   ├─ schedule3.py    the Schedule III head list and mapping rules
│   ├─ financials.py   the face of the statements, and derived figures
│   ├─ ratios.py       the eleven ratios and the 25 per cent rule
│   ├─ je_analytics.py the SA 240 routines and Benford
│   ├─ materiality.py  SA 320 materiality, SA 530 sampling
│   ├─ caro.py         CARO 2020 applicability and 21 clauses
│   ├─ formatting.py   Indian digit grouping
│   ├─ pipeline.py     sequencing
│   ├─ narrate.py      drafting, with an offline fallback
│   ├─ report.py       the eleven-sheet Excel workpaper
│   ├─ api.py          FastAPI
│   ├─ launch.py       desktop launcher - waits for the port, then opens the browser
│   └─ cli.py          command line
├─ web/               installable progressive web application
├─ automation/        two n8n workflows, exported
├─ samples/           synthetic client – clearly fictitious
├─ tests/             one file per area of statutory logic
└─ docs/              statutory basis, architecture, video script, recording guide (Markdown and PDF)
```

Where each day of AICA Level 2 appears

Day	Learning	Where it is used	Evidence
1	Agent architecture, advanced prompting	Memorandum, ratio notes, enquiry letter	<code>prompts/</code> — four versioned instructions with changelogs
2	Gemini API, system instructions, model parameters, local models	The drafting layer	<code>src/auditlens/narrate.py</code> — system instructions as files, temperature 0.2, LM Studio fallback
3	Python fundamentals and core libraries	The whole engine	<code>src/auditlens/</code> with 116 tests
4	Full-stack build, PWA, deployment	Dashboard and workpaper download	<code>web/</code> — manifest, service worker, installable
5	n8n workflow automation	Quarterly review; journal entry alert	<code>automation/</code> — two exported workflows

Limitations

Stated here rather than left for a reviewer to find:

- **Schedule III Division I only.** Ind AS (Division II) and NBFC presentation (Division III) are not implemented. The head list would need replacing; the engine around it would not.
- **It forms no opinion** and concludes on no CARO clause.
- **It does not determine materiality.** It computes it from the benchmark and rate the auditor chooses, and flags a rate outside the customary range.
- **It asserts no commercial reason** for any movement in a ratio. The prompt forbids it, and the draft carries a bracketed instruction to management instead.
- **It verifies nothing.** It analyses what the books say. Existence, completeness and valuation remain the auditor's to establish.
- **Net credit sales and purchases are not derivable from a trial balance.** The engagement team supplies the proportion; the default treats all revenue as credit sales and must be overridden where that is wrong.

Confidentiality

No client data is used anywhere in this project — not in the repository, not in the samples, and not in the demonstration video. Everything is built and demonstrated on synthetic data. Client information is covered by the confidentiality obligations of the Chartered Accountants Act, 1949 and the ICAI Code of Ethics, and a public repository is publication.

For engagements where confidentiality terms rule out a third-party API, run a local model in LM Studio. The analytical result is identical, because the model never touches a figure.

Disclaimer

Machine-generated analytical output. Every figure, selection and draft produced by this application requires the review and professional judgement of a Chartered Accountant before it is relied upon or issued. Nothing in this application constitutes an audit opinion, and nothing in it discharges any responsibility of the auditor under the Standards on Auditing.

Built by **CA. Rajendra Bagade**, Senior Partner, for AICA Level 2 Module C. Licensed under the MIT licence — see [LICENSE](#).

AuditLens · AICA Level 2 Module C capstone · CA. Rajendra Bagade · Machine-generated analytical output requires the review of a Chartered Accountant.